FINAL PROJECT REPORT

AI WEATHERWISE

Intelligent Weather Forecasting, AI Insights and Recommendations
Powered by Google Gemini AI

Project Type

AI-powered Weather REST API / Backend Application

Technology Stack

Node.js · Express.js · MongoDB Atlas · Mongoose · JWT · bcrypt.js · Google Gemini AI · OpenWeatherMap API

2. Abstract

Weather information is used every day for planning travel, work and outdoor activities, yet raw weather figures such as temperature, humidity and wind speed are not always easy to interpret, and users often have to visit different sources to find the cities they care about. AI WeatherWise is a backend REST API designed to address this problem. It is built with Node.js and Express.js and stores its data in MongoDB Atlas through Mongoose.

Users register and log in to the API. Protected routes are secured with JSON Web Tokens (JWT), and passwords are stored only as bcrypt hashes. Authenticated users can view their profile and manage a personal list of favorite cities / locations, with create, read, update and delete operations. The API retrieves real-time weather information from the OpenWeatherMap API and uses Google Gemini AI to generate weather summaries and recommendations that explain the weather data in simple language.

The project follows a layered architecture consisting of routes, authentication middleware, controllers, services and Mongoose models. It uses request validation, centralized error handling and a standard JSON response format, and it keeps secrets such as the MongoDB connection string, JWT secret and API keys in environment variables. AI WeatherWise is a backend/API project and was designed to be tested with Thunder Client / Postman.

3. Introduction

Weather information affects daily decisions such as what to wear, when to travel and how to schedule outdoor work. Weather data is available from many providers, but it usually arrives as raw metrics that require interpretation. A user who sees a temperature, a humidity percentage and a wind speed may still be unsure what they mean for the day ahead.

There is therefore a need for a system that not only retrieves weather data but also presents it in a simple and meaningful way. AI WeatherWise is a backend REST API that brings these pieces together. Any client that can send HTTP requests can use it; during development and testing, Thunder Client / Postman acts as the client.

3.1 How Users Interact with the API

A user first registers and logs in. After a successful login the API returns a JWT, which the user sends in the Authorization header of later requests. With a valid token the user can view their profile, manage favorite locations, request real-time weather information and request an AI-generated weather insight. Every response is returned as JSON.

3.2 Role of the Main Technologies

Table 1: Role of the main technologies

Technology	Role in AI WeatherWise
OpenWeatherMap API	External weather provider. Supplies real-time weather data for a requested location.
Google Gemini AI	Generates weather summaries and recommendations that explain the weather data in plain language.
MongoDB / MongoDB Atlas	Cloud database that stores user accounts and favorite location records.
Mongoose	Object modelling layer that defines the User and Location models and connects to MongoDB.
JWT authentication	Identifies the logged-in user and protects private routes.
bcrypt.js	Hashes passwords before storage and verifies them at login.
Express.js / Node.js	Runtime and web framework that hosts the REST API.

4. Problem Statement

The following problems motivate the development of AI WeatherWise:

- Raw weather information can be difficult for users to interpret.
- Weather information may need to be retrieved from external services rather than being available locally.
- Users need a structured way to manage frequently used cities.
- Weather-related recommendations can be difficult to understand from raw metrics alone.
- User-specific favorite locations need secure storage so that they are shown only to their owner.
- External API integrations can fail and therefore require proper error handling.
- Sensitive credentials and API keys must be protected and kept out of the source code.

5. Objectives

1. Provide secure user registration and login.
2. Implement JWT-based authentication for protected routes.
3. Allow users to manage favorite cities / locations.
4. Retrieve real-time weather information through the OpenWeatherMap API.
5. Generate AI-powered weather summaries and recommendations using Google Gemini.
6. Store user and favorite location data using MongoDB and Mongoose.
7. Validate API requests and provide clear JSON errors.
8. Provide consistent REST API responses.

6. Existing System

Traditional or manual ways of using weather information generally have the limitations listed below. This section makes no claims about any particular commercial application.

- Users depend on raw weather data that they must interpret themselves.
- Information may be spread across different sources.
- Raw weather metrics may require interpretation before they can be acted upon.
- Frequently used cities may not be conveniently managed in one place.
- There may be no personalized AI explanation of the weather.
- Failures of external services need to be handled properly rather than passed on to the user as raw errors.

7. Proposed System

AI WeatherWise provides a secure, API-based solution. Users create an account and log in to receive a JWT. With that token they can manage favorite locations stored in MongoDB Atlas, each linked to its owner. They can retrieve real-time weather information, which the API obtains from OpenWeatherMap, and they can request AI output in the form of weather summaries and recommendations generated by Google Gemini.

Passwords are protected with bcrypt hashing, protected routes are guarded by JWT authentication middleware, requests are validated, all errors pass through a centralized error handler, and every response follows a standard JSON format. Secrets (the MongoDB connection string, the JWT secret and the API keys) are held in environment variables.

Table 2: Summary of proposed system features

Feature	Description
Secure authentication	Registration and login with bcrypt hashing and JWT-protected routes.
Favorite location management	Create, read, update and delete favorite cities/locations for the logged-in user.
Real-time weather	Weather data retrieved from OpenWeatherMap.
AI summaries and recommendations	Weather explanation and recommendations generated by Google Gemini.
Validation and error handling	Input validation, centralized error middleware and clear JSON errors.
Standard JSON responses	Consistent response format across all endpoints.

8. Requirements

8.1 Functional Requirements

The functional requirements listed below correspond to the features of the AI WeatherWise API. Exact operation names should be checked against the implemented project (To be verified against the implemented project.).

- Register a new user account and log in with email and password.
- Issue a JWT on login and verify it on protected routes.
- Return the authenticated user's profile.
- Add a favorite location for the authenticated user.
- List the authenticated user's favorite locations.
- View, update and delete a favorite location, where implemented in the project.
- Fetch real-time weather data from OpenWeatherMap.
- Generate an AI weather summary / recommendation using Google Gemini.
- Validate input and return clear JSON error responses.

8.2 Non-Functional Requirements

Table 3: Non-functional requirements

Requirement	Description
Security	Passwords stored as bcrypt hashes; JWT-protected routes; secrets kept in environment variables.
Reliability	Centralized error handling gives consistent handling of failures.
Maintainability	Modular structure with separate routes, middleware, controllers, services and models.
Scalability	Stateless JWT authentication and a cloud database allow the API to be extended and deployed in different environments.
API consistency	Standard JSON response format for all endpoints.
Validation	Request input is checked before it is processed.
Error handling	All errors are returned as JSON with an appropriate status code.
External API availability	Weather retrieval and AI output depend on OpenWeatherMap and Google Gemini being reachable.
Database availability	User and location data depend on MongoDB Atlas being reachable.

8.3 Hardware Requirements

Table 4: Hardware requirements

Resource	Minimum	Recommended
Computer / laptop	Dual-core 64-bit processor	Quad-core processor
Processor	Dual-core, 2.0 GHz or higher	Quad-core, 2.5 GHz or higher

Resource	Minimum	Recommended
RAM	4 GB	8 GB or more
Storage	10 GB free	256 GB SSD, 20 GB or more free
Internet connection	Stable connection (needed for MongoDB Atlas, OpenWeatherMap, the Gemini API, npm and GitHub)	Stable broadband connection

8.4 Software Requirements

Table 5: Software requirements

Software	Purpose
Windows / Linux / macOS	Operating system
Node.js (LTS) and npm	Runtime and package manager
Visual Studio Code (or equivalent)	Code editor
MongoDB Atlas	Cloud database
Thunder Client / Postman	API testing
Git / GitHub	Version control
OpenWeatherMap API access	Real-time weather data (API key required)
Google Gemini API access	AI summaries and recommendations (API key required)

Development technologies: JavaScript, Node.js, Express.js, Mongoose, JWT, bcrypt.js, dotenv, CORS, the OpenWeatherMap API and the Google Gemini API.

9. System Design

9.1 Technical Architecture

AI WeatherWise uses a layered architecture. A request from the client passes through the Express.js server, the API routes and the authentication middleware to a controller. The controller calls a service, which uses the Mongoose models to reach MongoDB Atlas. For weather requests the service calls the OpenWeatherMap API, and for AI requests it uses the Gemini AI service, which calls the Google Gemini API. Figure 1 shows the flow.

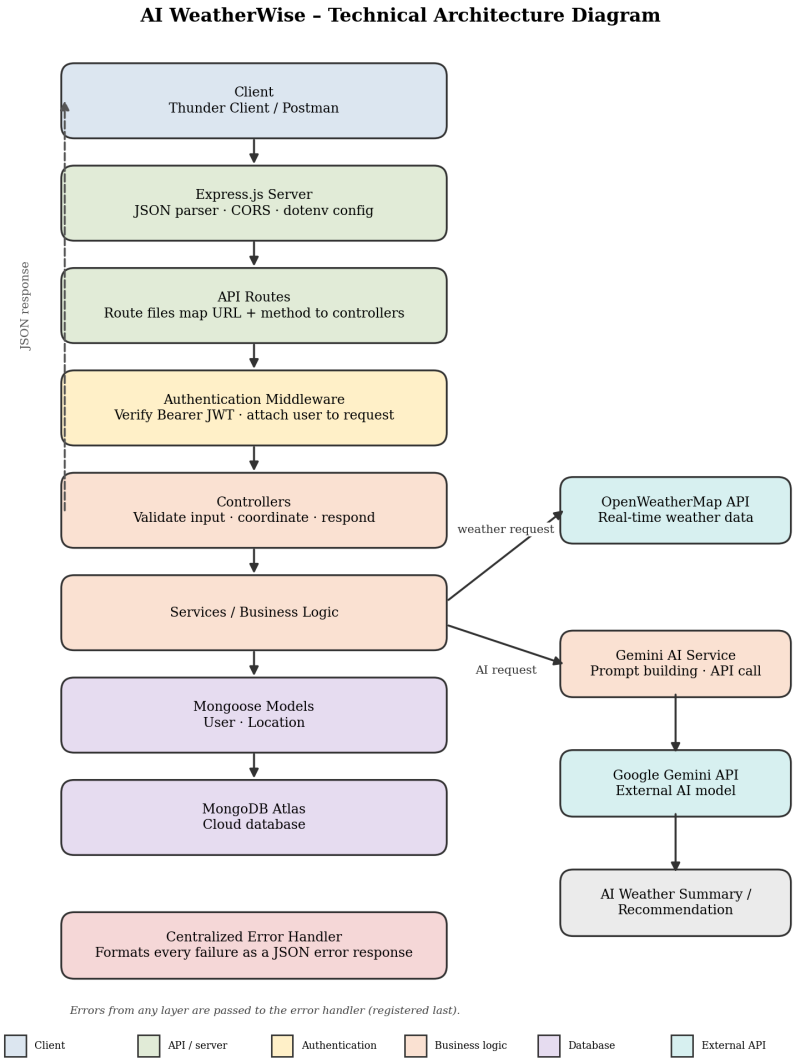


Figure 1: AI WeatherWise – Technical Architecture Diagram

Table 6: Architecture components and responsibilities

Component	Responsibility
Client	Thunder Client / Postman (or any HTTP client) sends requests and receives JSON responses.
Express.js server	Receives HTTP requests, applies JSON parsing and CORS, and mounts the routes.
API routes	Map URLs and HTTP methods to controller functions.

Component	Responsibility
Authentication middleware	Verifies the Bearer JWT and attaches the user to the request.
Controllers	Validate input, call services and return JSON responses.
Services	Hold the business logic, including weather retrieval and AI generation.
Mongoose models	Define the User and Location data structures.
MongoDB Atlas	Cloud database storing users and favorite locations.
OpenWeatherMap API	External provider of real-time weather data.
Gemini AI service / Google Gemini API	Builds the prompt and obtains the AI weather summary / recommendation.
Error middleware	Catches errors and returns a consistent JSON error response.

The weather flow is: Controller / Service → OpenWeatherMap API → real-time weather data → response. The AI flow is: weather data / validated request → Gemini AI service → Google Gemini API → AI weather summary / recommendation → controller → JSON response.

9.2 Database / ER Design

The database has two collections, User and Location. One User has many favorite Locations; the user field of a Location stores the _id of its owner (a Mongoose reference to User). No other entities are used. The fields shown are those recorded for the project; confirm them against the implemented Mongoose models (To be verified against the implemented project.).

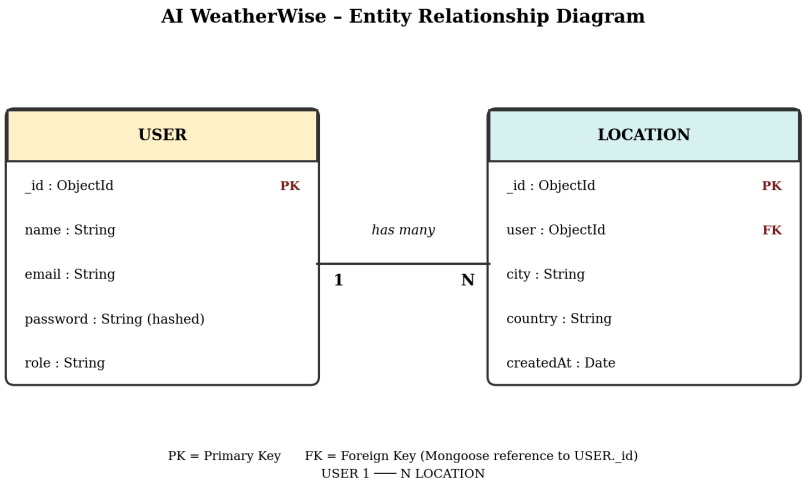


Figure 2: AI WeatherWise – Entity Relationship Diagram

Table 7: Database entities and fields

Entity	Field	Type	Notes
USER	_id	ObjectId	Primary key
USER	name	String	User's name
USER	email	String	User's login email

Entity	Field	Type	Notes
USER	password	String	Stored as a bcrypt hash
USER	role	String	User role
LOCATION	_id	ObjectId	Primary key
LOCATION	user	ObjectId	Reference to USER._id (owner)
LOCATION	city	String	Favorite city name
LOCATION	country	String	Country of the city
LOCATION	createdAt	Date	Creation date

9.3 User Flow

A new user registers, then logs in to obtain a JWT. The token is sent with each protected request. If it is valid, the user can view the profile, manage favorite locations, fetch weather or request an AI weather insight. The request then passes through the controller and service to MongoDB Atlas, OpenWeatherMap or Google Gemini, and a standard JSON response is returned. Errors at any stage are handled by the centralized error middleware and returned as JSON.

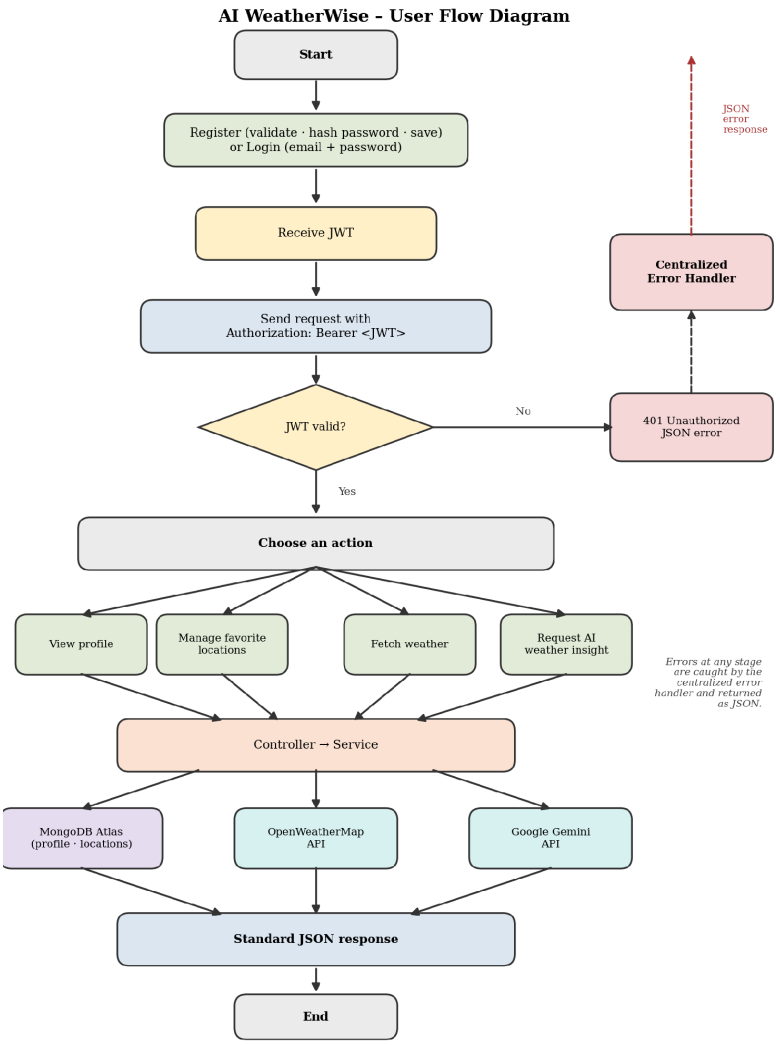


Figure 3: AI WeatherWise – User Flow Diagram

9.4 MVC Architecture

The code follows the Model–View–Controller idea, adapted to a REST API. The Model consists of the Mongoose User and Location models that represent data in MongoDB Atlas. The Controller receives the request, validates input, coordinates operations and returns responses. Because AI WeatherWise has no user interface, the View is the JSON response returned to the client, not an HTML page.

Routes, authentication middleware and services are additional layers around MVC: routes direct requests to controllers, the middleware protects them, and services hold the business logic, including the weather retrieval logic and the Gemini AI service.

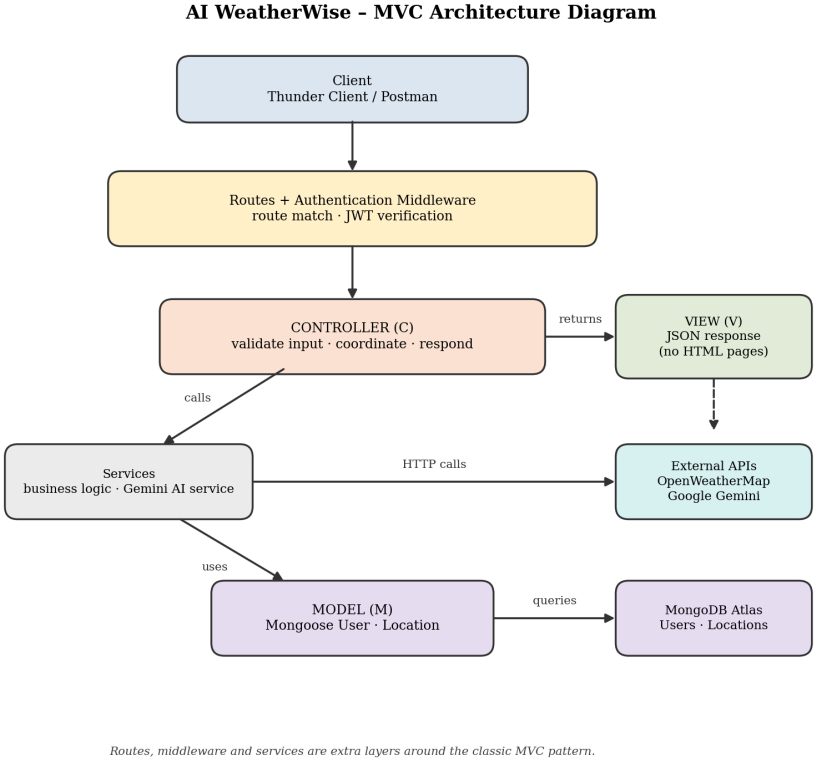


Figure 4: AI WeatherWise – MVC Architecture Diagram

10. System Features

Table 8: System features

No.	Feature	Description
1	Secure registration	Creates a user account after validating input; the password is hashed before saving.
2	Login	Verifies the email and password and returns a JWT.
3	JWT authentication	Protected routes accept only requests carrying a valid Bearer token.
4	Password hashing	bcrypt.js hashes passwords; plain-text passwords are never stored.
5	Protected routes	Authentication middleware guards user-specific operations.
6	Favorite location management	Users manage their own list of favorite cities/locations.
7	Real-time weather integration	Weather data is retrieved from the OpenWeatherMap API.
8	AI weather summaries	Google Gemini generates a plain-language summary of the weather.
9	AI recommendations / insights	Google Gemini generates weather-related recommendations.
10	MongoDB storage	Users and locations are stored in MongoDB Atlas through Mongoose.
11	Validation	Request input is checked before processing.
12	Error handling	Centralized error middleware returns consistent JSON errors.
13	Standard JSON responses	All endpoints return responses in one consistent structure.
14	Environment-based secrets	Connection strings, JWT secret and API keys are kept in environment variables.

11. Implementation

The implementation of AI WeatherWise is summarized below. It describes the architecture of the project and should be checked against the actual project code (To be verified against the implemented project.).

11.1 Express.js Backend

The server entry file loads the environment variables with dotenv, connects to MongoDB Atlas and starts the server on port 5001. The Express application applies the JSON body parser and CORS, mounts the route files and registers the centralized error handler last so that it can catch errors raised anywhere in the request pipeline.

11.2 Routes, Middleware, Controllers and Services

- **Routes:** map URLs and HTTP methods for authentication, profile, location, weather and AI functions to controller functions.
- **JWT authentication middleware:** reads the Bearer token, verifies it with the JWT secret and attaches the user to the request.
- **Controllers:** read and validate input, call services and send the JSON response.
- **Services:** contain business logic such as retrieving weather data and generating AI output.
- **Error middleware:** formats all errors into a consistent JSON error response.

11.3 Mongoose Models and MongoDB

The User model holds _id, name, email, password (stored as a hash) and role. The Location model holds _id, user (a reference to the owning User), city, country and createdAt. One User can own many Locations. Mongoose connects to MongoDB Atlas using the connection string from the environment and applies schema-level validation. Passwords are stored only in hashed form.

11.4 bcrypt.js and JWT

Before a user is saved, the password is hashed with bcrypt.js. At login, the entered password is compared with the stored hash. After a successful login, a JWT signed with the JWT secret is returned to the client. The authentication middleware verifies this token on protected routes and rejects requests whose token is missing or invalid.

11.5 OpenWeatherMap Integration

For a weather request, the controller validates the input and passes it to the weather service. The service sends a request to the OpenWeatherMap API using the API key held in the environment, receives the real-time weather data and shapes it into the API's JSON response. If OpenWeatherMap cannot be reached, or returns an error (for example for an unknown location), the failure is passed to the centralized error handler.

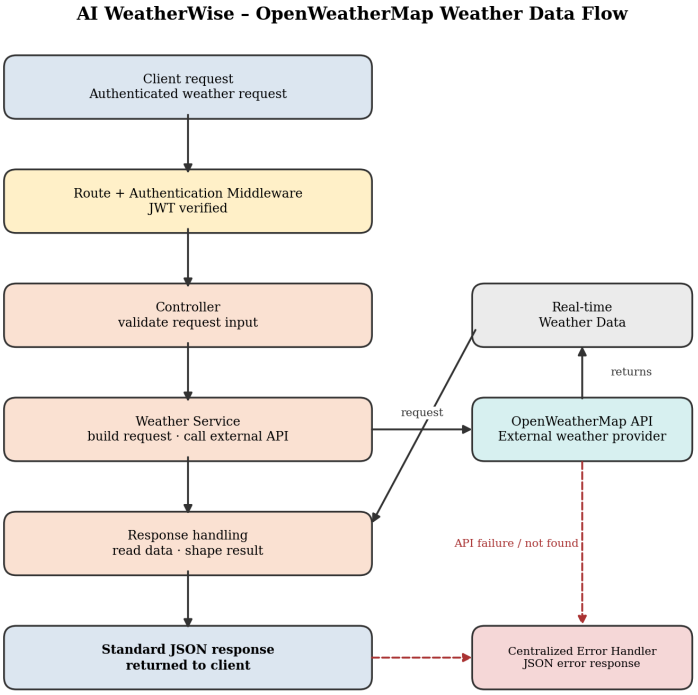


Figure 5: AI WeatherWise – OpenWeatherMap Weather Data Flow

11.6 Google Gemini AI Service

The Gemini AI service receives validated weather-related input or weather data from the controller, builds a prompt, sends it to the Google Gemini API with the API key from the environment, and returns the generated summary or recommendation to the controller, which sends it in the standard JSON format. Failures of the Gemini service are passed to the centralized error handler, so that no API key or stack trace is exposed to the client.

> *The exact prompt text and the exact input fields for AI requests are not reproduced in this report. To be verified against the implemented project.*

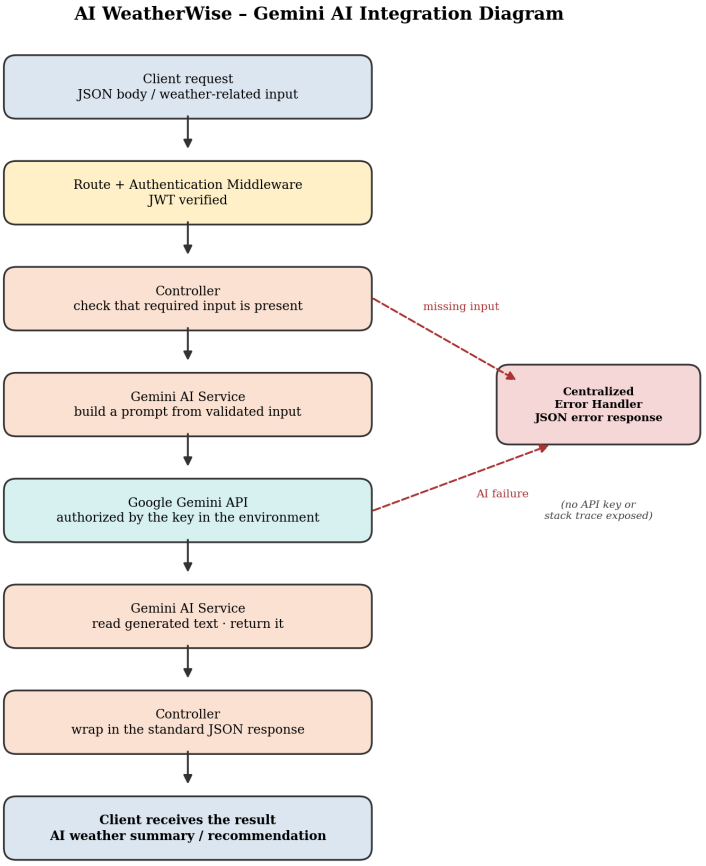


Figure 6: AI WeatherWise – Gemini AI Integration Diagram

12. API Documentation Summary

Full details for each group of endpoints are in Part B. The server runs locally at http://localhost:5001. The HTTP method shown reflects the standard REST operation for each purpose; the endpoint path of each operation must be confirmed. Where an endpoint cannot be confirmed from the project source, the table states: “Endpoint should be verified against the implemented project code”.

Table 9: API documentation summary

Method	Endpoint	Purpose	Authentication	Input
Authentication APIs				
POST	Endpoint should be verified against the implemented project code	Register a new user	None	name, email, password (to be verified)
POST	Endpoint should be verified against the implemented project code	Log in and receive a JWT	None	email, password
Profile APIs				
GET	Endpoint should be verified against the implemented project code	Get the authenticated user's profile	Bearer JWT	None
Location APIs				
POST	Endpoint should be verified against the implemented project code	Add a favorite location	Bearer JWT	city, country (to be verified)
GET	Endpoint should be verified against the implemented project code	List favorite locations	Bearer JWT	None
GET	Endpoint should be verified against the implemented project code	View one favorite location (where implemented)	Bearer JWT	Location ID
PUT	Endpoint should be verified against the implemented project code	Update a favorite location (where implemented)	Bearer JWT	Location ID, fields to update
DELETE	Endpoint should be verified against the implemented project code	Delete a favorite location (where implemented)	Bearer JWT	Location ID
Weather APIs				
GET	Endpoint should be verified against the implemented project code	Fetch real-time weather data	To be verified	City / location (to be verified)
AI APIs				
POST / GET	Endpoint should be verified against the implemented project code	Generate an AI weather summary / recommendation	To be verified	Weather-related input (to be verified)

13. Testing

The API is tested manually with Thunder Client / Postman using black-box test cases against the local server at http://localhost:5001. Protected routes are tested with the header Authorization: Bearer <JWT>. The test plan covers the areas below.

Table 10: Test plan summary

Test area	What is checked	Expected result	Status
Authentication	Registration, login, JWT issue, access with and without a token	Success on valid data; 401 for missing/invalid token	To be recorded during execution
Weather API	Fetching real-time weather for a valid and an invalid location	Weather data for valid input; JSON error for invalid input	To be recorded during execution
Favorite Location CRUD	Add, list, view, update and delete locations	Operations succeed only for the owner's data	To be recorded during execution
AI testing	AI summary / recommendation with valid and incomplete input	Generated text for valid input; error for missing input	To be recorded during execution
Validation testing	Missing or malformed fields in requests	Clear JSON validation error	To be recorded during execution
Error handling testing	Unknown routes, external API failure, invalid IDs	Consistent JSON error response	To be recorded during execution

> *Status of testing: the test cases are defined, but their actual results must be recorded during execution. This report therefore does not state how many tests passed or failed. After execution, insert the final Passed / Failed counts here.*

PART B – API DOCUMENTATION

B.1 API Conventions

Table 12: API conventions

Item	Description
Base URL	http://localhost:5001
Data format	Requests and responses use JSON (Content-Type: application/json).
Authentication	Protected endpoints need the header Authorization: Bearer <your-jwt-token>.
Response format	A standard JSON structure is used by all endpoints. The structure shown in this document is illustrative; check the exact fields against the implementation.
Errors	Returned by the centralized error handler as JSON with an appropriate HTTP status code.
Placeholders	All values in the examples are placeholders, not real data.

> *Endpoint paths are not listed in this report because they could not be confirmed from the project source. Each endpoint below is marked: "Endpoint should be verified against the implemented project code".*

B.2 Request Format

```
Content-Type: application/json
Authorization: Bearer <your-jwt-token>   (protected endpoints only)

{
  "field": "value"
}
```

B.3 Response Format (illustrative)

```
{
  "success": true,
  "message": "<message>",
  "data": { }
}
```

B.4 Error Format (illustrative)

```
{
  "success": false,
  "message": "<error description>"
}
```

Table 13: Common HTTP status codes

Status code	Typical meaning
200 / 201	Request succeeded / resource created
400	Validation error or missing input

Status code	Typical meaning
401	Missing or invalid JWT, or invalid credentials
404	Resource not found
500 / 502	Server error or external API failure (to be verified)

B.5 Authentication APIs

Method	Endpoint	Purpose	Auth	Input
POST	Endpoint should be verified against the implemented project code	Register a new user	None	name, email, password (to be verified)
POST	Endpoint should be verified against the implemented project code	Log in; returns a JWT	None	email, password

B.6 Profile APIs

Method	Endpoint	Purpose	Auth	Input
GET	Endpoint should be verified against the implemented project code	Return the authenticated user's profile	Bearer JWT	None

B.7 Favorite Location APIs

Method	Endpoint	Purpose	Auth	Input
POST	Endpoint should be verified against the implemented project code	Add a favorite location	Bearer JWT	city, country (to be verified)
GET	Endpoint should be verified against the implemented project code	List favorite locations	Bearer JWT	None
GET	Endpoint should be verified against the implemented project code	View one location (where implemented)	Bearer JWT	Location ID
PUT	Endpoint should be verified against the implemented project code	Update a location (where implemented)	Bearer JWT	Location ID, fields
DELETE	Endpoint should be verified against the implemented project code	Delete a location (where implemented)	Bearer JWT	Location ID

PART C – INSTALLATION AND USER GUIDE

C.1 Prerequisites

- A computer running Windows, Linux or macOS with an internet connection.
- A code editor such as Visual Studio Code.
- A MongoDB Atlas account, an OpenWeatherMap API key and a Google Gemini API key.
- Thunder Client (VS Code extension) or Postman.
- Git (optional, for cloning the project).

C.2 Install Node.js

Download the LTS version from the official Node.js website and install it. npm is installed together with Node.js. Check the installation:

```
node -v
npm -v
```

C.3 Install Dependencies

Open a terminal in the project folder and install the project dependencies listed in package.json:

```
npm install
```

C.4 Configure Environment Variables

Create a file named .env in the project root folder. Use your own values in place of the placeholders. Never share this file or commit it to GitHub. Use the variable names your code actually reads; the names below are examples and must match the project's configuration (To be verified against the implemented project.).

```
PORT=5001
MONGODB_URI=<your-mongodb-atlas-connection-string>
JWT_SECRET=<your-long-random-secret>
OPENWEATHER_API_KEY=<your-openweathermap-api-key>
GEMINI_API_KEY=<your-gemini-api-key>
```

C.5 Configure MongoDB Atlas

1. Create a MongoDB Atlas account and a cluster.
2. Create a database user with a username and password.
3. In Network Access, allow your IP address.
4. Use the cluster's Connect option to copy the connection string, replace the password placeholder with the database user's password, and store it in the MongoDB variable of the .env file.

C.6 Configure OpenWeatherMap API

Create an account on the OpenWeatherMap website, generate an API key by following the OpenWeatherMap API documentation (see Part D), and store it in the weather API key variable of the .env file. Keep the key private.

C.7 Configure Gemini API

Create a Gemini API key by following the Google Gemini API documentation (see Part D), and store it in the Gemini API key variable of the .env file. Keep the key private.

C.8 Start the Server

```
npm run dev    # development mode, if a dev script exists
npm start      # standard start, if a start script exists
```

Use the script defined in the project's package.json. On success the console shows that the server is running on port 5001 and that MongoDB is connected (the exact messages depend on the code).

C.9 Test the Root API

In Thunder Client / Postman, send GET http://localhost:5001/. A successful reply confirms that the server is running; the response text depends on the project code.

Using Thunder Client / Postman

1. Create a new request and choose the HTTP method (GET, POST, PUT or DELETE).
2. Enter the URL, for example http://localhost:5001/<endpoint-path>.
3. For POST and PUT, open Body → JSON and enter the request body.
4. For protected endpoints, open Auth → Bearer (or the Headers tab) and paste the token from the login response.
5. Click Send and check the status code and JSON response.
6. Optionally save the token as an environment variable (for example token) and use it as {{token}}.

C.10 Authentication Testing

1. Send the register request with the required user fields. Expect a success status (201 is typical).
2. Send the login request with the same email and password. Expect 200 and a token. Copy the token.
3. Send the profile request with the Bearer token. Expect 200 and your profile. Without the token, expect 401.

Final Conclusion

This documentation consolidates the requirements, design, implementation, testing and usage information of AI WeatherWise into a structured final project document. It presents the system as a REST API backend with authentication, favorite location management, real-time weather retrieval and Gemini AI weather summaries and recommendations. It also provides the API reference structure, the installation guide and the references needed to understand, run and test the project.

Results that depend on running the project (test outcomes, screenshots and confirmed endpoint paths) are marked to be recorded or verified during execution, so that the final report contains only genuine evidence.